Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

Clase # 9 - PAO 2026 – II

Agenda

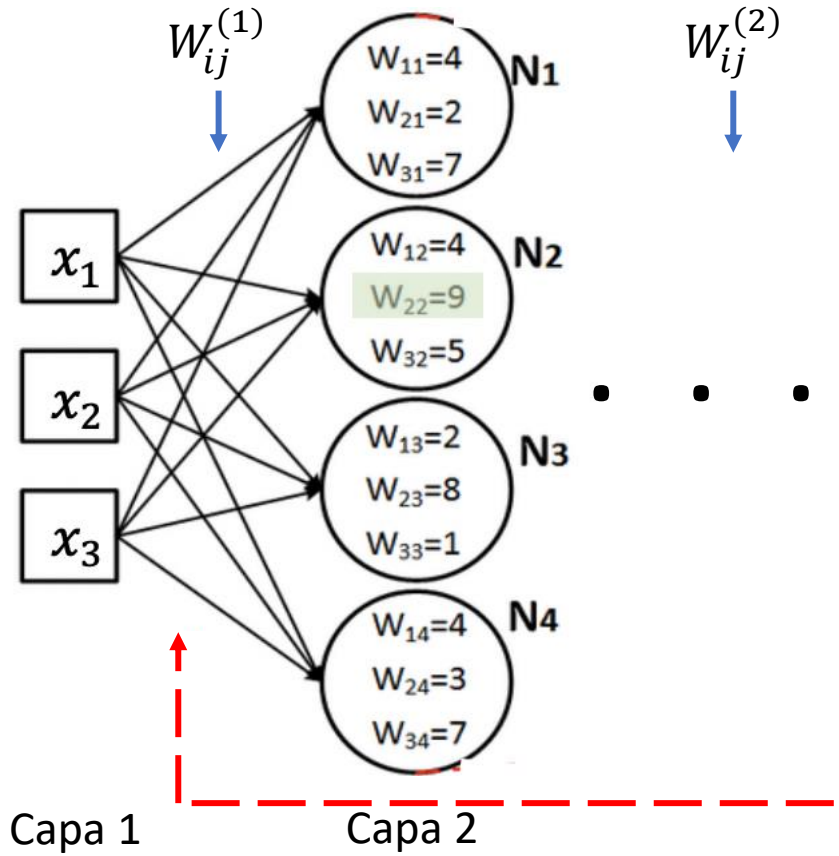
1. Aprendizaje en las redes MLP
   - 1.1 Otro ejemplo: Entrenamiento en redes MLP
   - 1.2 Aprendizaje en red – forward propagation
   - 1.3 Backpropagation en MLP
   - 1.4 Entrada / Salida / Funciones de Umbral y de Error en MLP

Redes Neuronales

Pesos

- Las RN pueden tener pesos fijos y adaptables
- Las RN con pesos adaptables utilizan leyes de aprendizaje para ajustar los valores de la conexión con otras neuronas
- Una vez que la RN aprende los parámetros (pesos), se fijan para realizar la tarea para la cual fue diseñada la red
- Los pesos adaptables son los que definen la función que estamos buscando con la RN

Redes Neuronales



W_{ij}^{k}

	N_1	N_2	N_3	N_4
x_0	W_{01}	W_{02}	W_{03}	W_{04}
x_1	$W_{11}=4$	$W_{12}=4$	$W_{13}=4$	$W_{14}=4$
x_2	$W_{21}=4$	$W_{22}=4$	$W_{23}=4$	$W_{24}=4$
x_3	$W_{31}=4$	$W_{32}=4$	$W_{33}=4$	$W_{34}=4$

$$W^{(k)} = L_{k+1} \times (L_k + 1)$$

$$W^{(1)} = L_{1+1=2} \times (L_1 + 1) = 4 \times 4$$

- Esta matriz se llama W^1. Cuando hay más capas, tenemos matrices de pesos, W^2, W^3, etc.
- En general, si una capa L tiene M neuronas y la siguiente capa $L + 1$ tiene N neuronas, la matriz de pesos es una matriz $N \times M$ (N filas y M columnas)

Redes Neuronales

Entrenamiento

- Buscamos una estrategia iterativa que permita a la neurona adapte sus parámetros (pesos) de conexión, cuando se presentan los datos de entrenamiento en la entrada de la red
- El entrenamiento del perceptrón se basa en una ley de aprendizaje general, conocida como la regla delta:
  - Para cada peso W se realiza un ajuste basado en un ΔW, y de acuerdo con la siguiente regla:

$$\Delta W_{ij} = \alpha (T_j - Y_j) Y_i$$

Donde:

α es la velocidad de aprendizaje. Un factor entre $0 < \alpha \leq 1$

Y_i valor de salida de la neurona i (o la entrada a la neurona a partir de la segunda capa)

T_j valor deseado, esperado o target conocido en la neurona j

Y_j valor de salida calculado por la neurona j

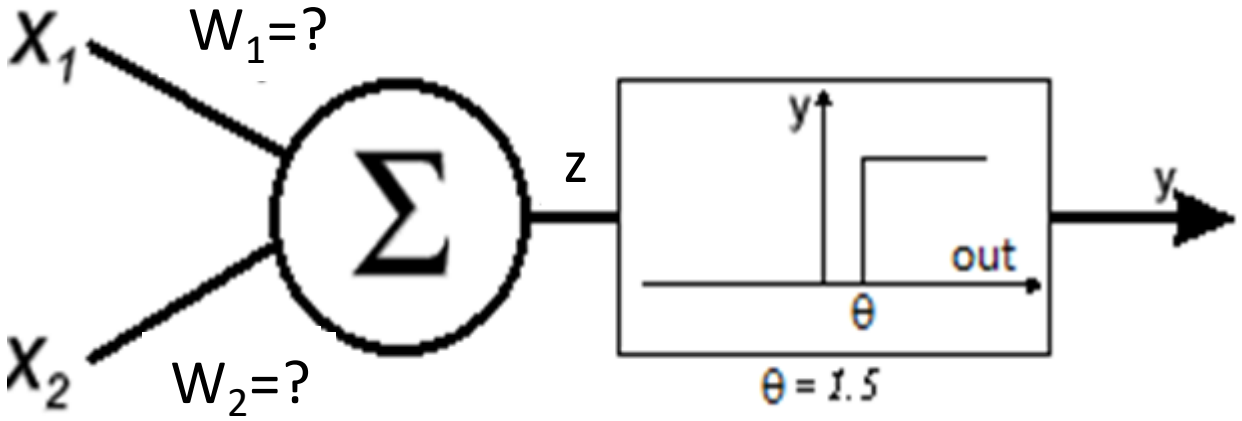
Aprendizaje

- Seleccionar α
- Inicializar los pesos con valores aleatorios (W)
- Mientras todos los ejemplos no sean clasificados correctamente
  - Ingresar un nuevo ejemplo a la red
  - Si el ejemplo fue incorrectamente clasificado
    - $W_{nuevo} = W_{anterior} + \alpha\,(t - y)\,x$

Aprendizaje: Ejemplo

- Entrenar un perceptron para que reconozca el patrón de comportamiento de la puerta AND
- Supongamos los siguientes parámetros:

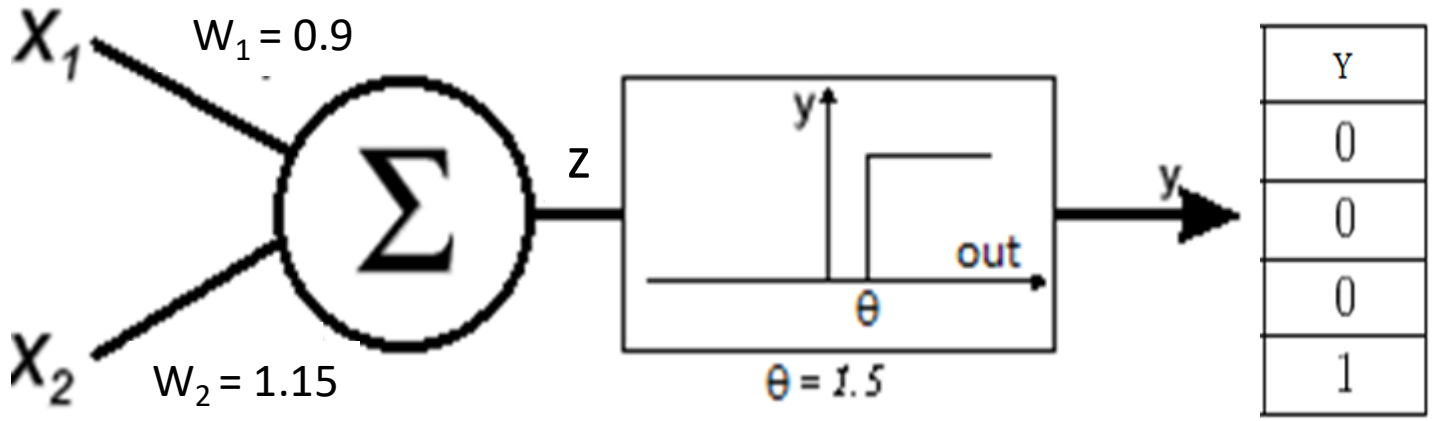
$\alpha = 0.3$
$\theta = 1.5$
$W_1 = 0$
$W_2 = 0.25$



Aprendizaje: Ejemplo

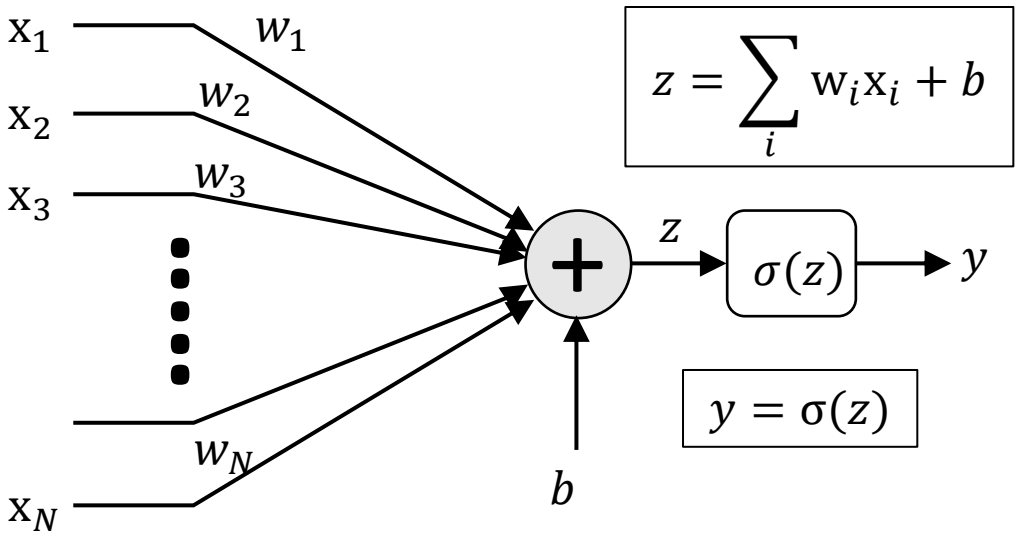
$\alpha = 0.3$
$\theta = 1.5$
$W_1 = 0$
$W_2 = 0.25$

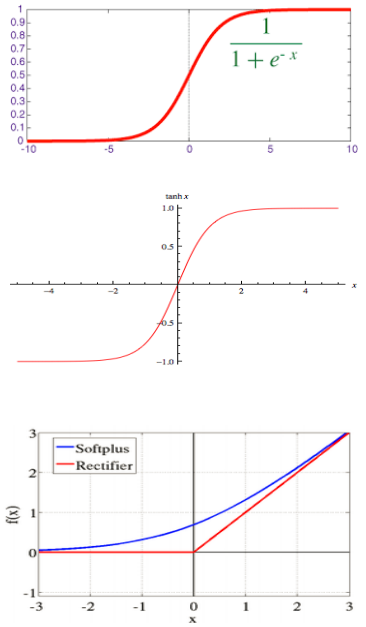
x_1	x_2
0	0
0	1
1	0
1	1



Y
0
0
0
1

Aprendizaje en red





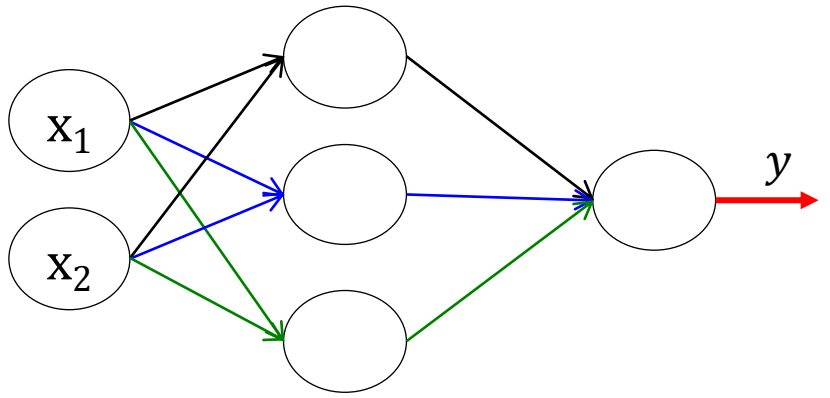
Funciones de activación $\sigma(z)$

- Recordar que tenemos redes de neuronas
- Veamos cómo se propaga la salida de la neurona a las siguientes neuronas en la red

Propagación hacia delante - forward propagation

- ✓ Considere la siguiente red de 3 capas y 4 unidades de activación sigmoidal (neuronas con una función de umbral tipo sigmoidal)
- ✓ Entrenar la red y determinar la salida de la red utilizando los patrones de entrenamiento de la siguiente tabla

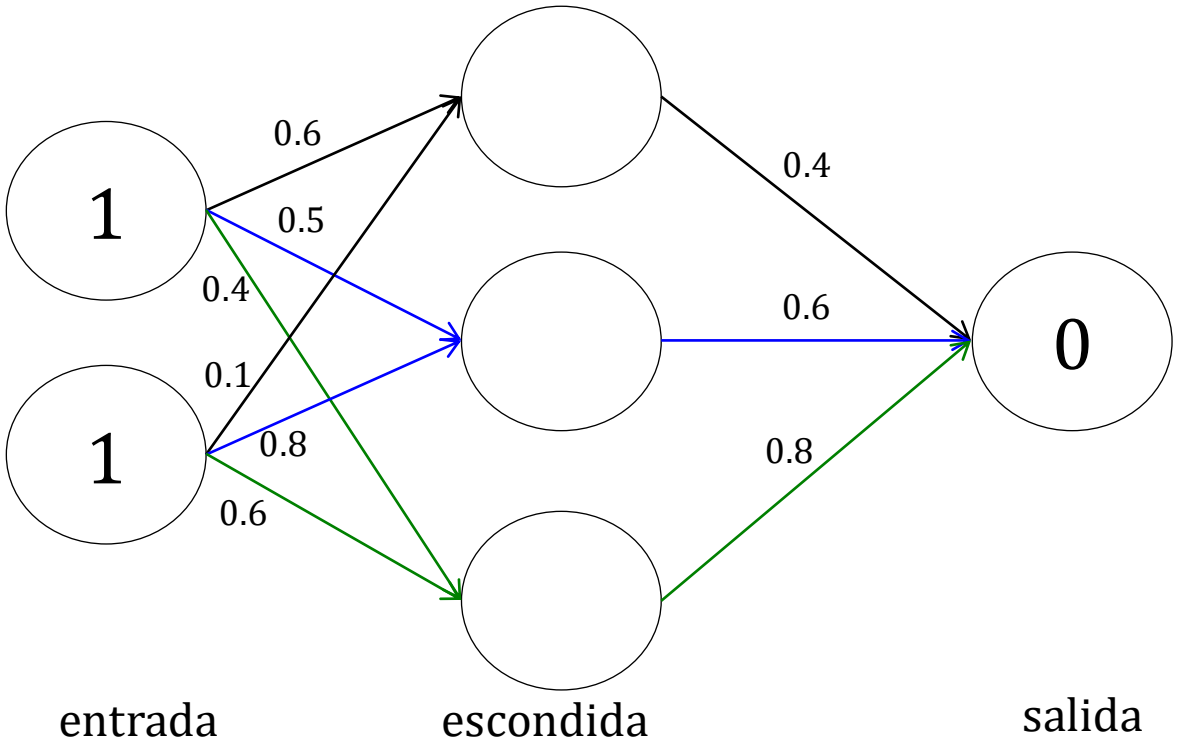
x_1 x_2	y
0, 0	0
0, 1	1
1, 0	1
1, 1	0



Para simplificar los cálculos, vamos a omitir el bias

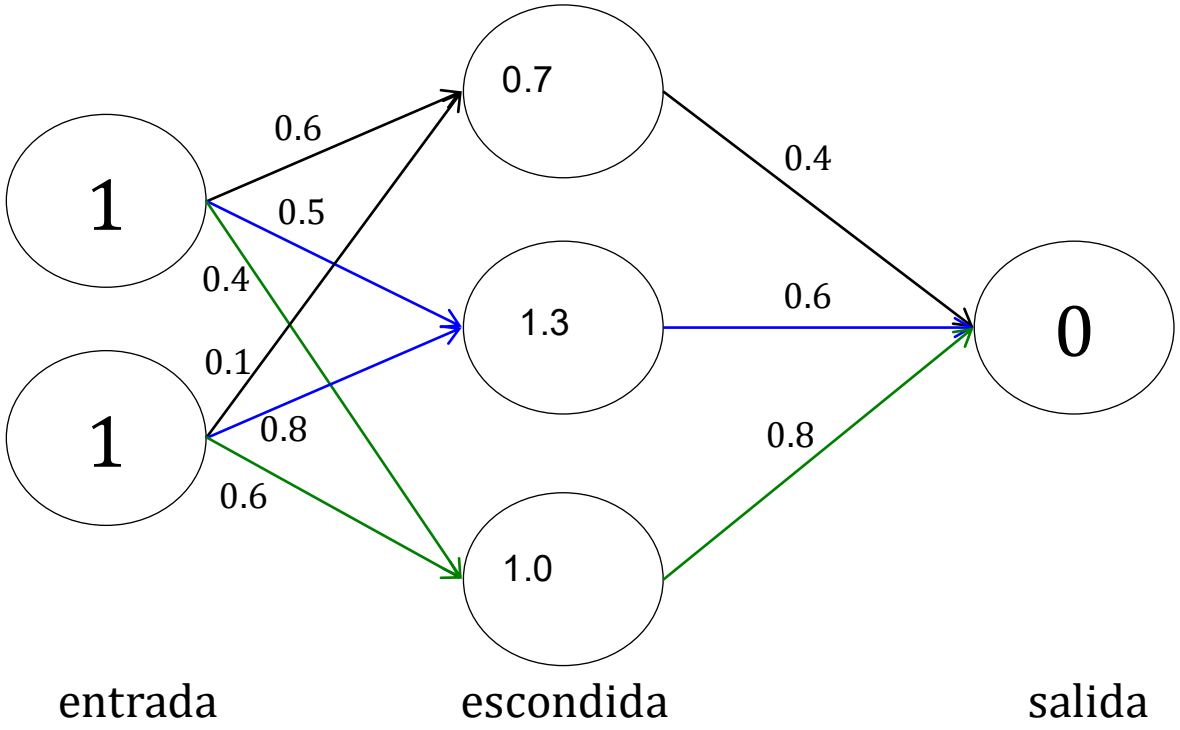
Propagación hacia delante - forward propagation

- ✓ Al inicio asignamos los pesos de forma aleatoria, generalmente entre 0 y 1, pero los pesos finales no necesariamente lo son



Propagación hacia delante - forward propagation

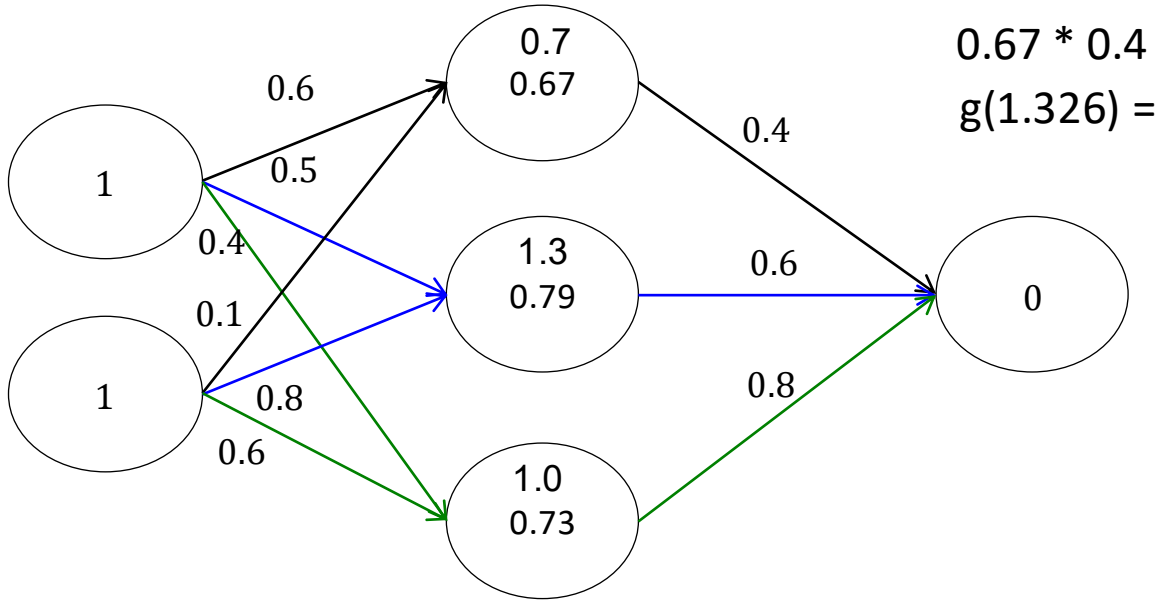
- ✓ Suma del producto de las entradas con sus respectivos pesos ingresan a la capa escondida:



1 * 0.6 + 1 * 0.1 = 0.7
1 * 0.5 + 1 * 0.8 = 1.3
1 * 0.4 + 1 * 0.6 = 1.0

Propagación hacia delante - forward propagation

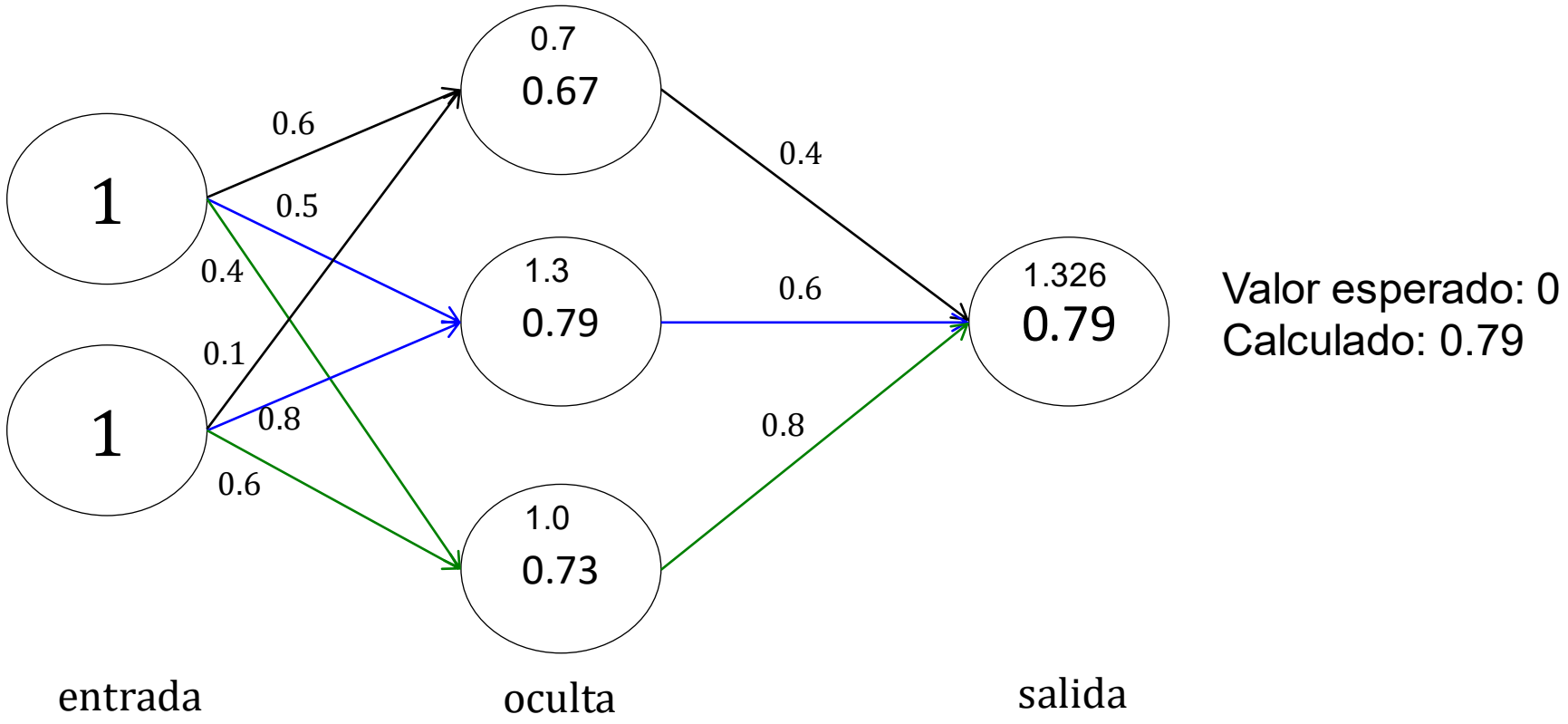
- ✓ Igual que antes, sumamos el producto de la activación de las capas escondidas, con los pesos de la segunda capa (que fueron determinados aleatoriamente en la primera iteración)
- ✓ Y, aplicamos la función de activación en la capa de salida.



0.67 * 0.4 + 0.79 * 0.6 + 0.73 * 0.8 = 1.326

g(1.326) = 0.790178218

Propagación hacia delante - forward propagation



Valor esperado: 0
Calculado: 0.79

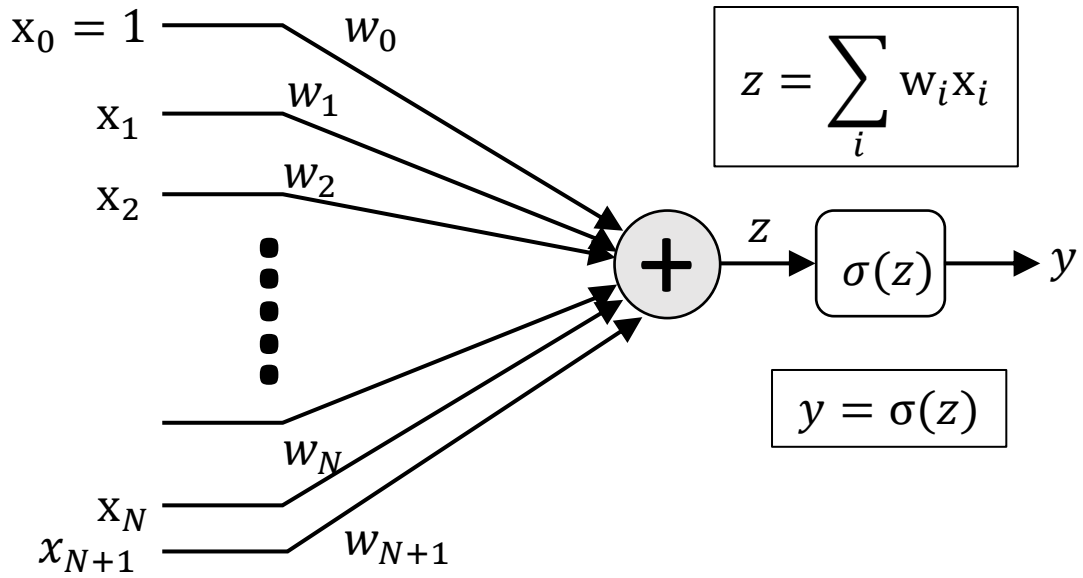
Propagación hacia delante - forward propagation

- ✓ Debido a que usamos un conjunto aleatorio de pesos iniciales, el valor total en la neurona de salida está errado: +0.79 (y el objetivo era 0)
- ✓ Si detenemos aquí el aprendizaje, este conjunto de pesos no servirá para reconocer los patrones de la entrada (operación XOR)
- ✓ Cómo calcular el error entre el valor esperado (0) y la salida real de la red (+0.79)?
- ✓ Luego de calcular el error, podemos aplicar la regla de aprendizaje Delta para aprender (corregir / descubrir) los pesos de la red, utilizando:

$$w_{nuevo} = w_{anterior} + \alpha(t - y)x$$

Analicemos este término

Aprendizaje en red



- El bias también se puede ver como el peso de otro componente de entrada que está fijo en 1
- Notar que: Si el bias no se menciona de manera explícita, siempre está. Recuerde que cada perceptron siempre tiene una entrada adicional (bias) que está fija en 1